Informatica® Cloud Application Integration
May 2025

AI Agent with Multifunction Calling using Google Vertex AI

Informatica Cloud Application Integration AI Agent with Multifunction Calling using Google Vertex AI
May 2025

© Copyright Informatica LLC 2025

This software and documentation contain proprietary information of Informatica LLC and are provided under a license agreement containing restrictions on use and disclosure and are also protected by copyright law. Reverse engineering of the software is prohibited. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC. This Software may be protected by U.S. and/or international Patents and other Patents Pending.

Use, duplication, or disclosure of the Software by the U.S. Government is subject to the restrictions set forth in the applicable software license agreement and as provided in DFARS 227.7202-1(a) and 227.7702-3(a) (1995), DFARS 252.227-7013©(1)(ii) (OCT 1988), FAR 12.212(a) (1995), FAR 52.227-19, or FAR 52.227-14 (ALT III), as applicable.

The information in this product or documentation is subject to change without notice. If you find any problems in this product or documentation, please report them to us in writing.

Informatica, Informatica Platform, Informatica Data Services, PowerCenter, PowerCenterRT, PowerCenter Connect, PowerCenter Data Analyzer, PowerExchange, PowerMart, Metadata Manager, Informatica Data Quality, Informatica Data Explorer, Informatica B2B Data Transformation, Informatica B2B Data Exchange Informatica On Demand, Informatica Identity Resolution, Informatica Application Information Lifecycle Management, Informatica Complex Event Processing, Ultra Messaging, Informatica Master Data Management, and Live Data Map are trademarks or registered trademarks of Informatica LLC in the United States and in jurisdictions throughout the world. All other company and product names may be trade names or trademarks of their respective owners.

Portions of this software and/or documentation are subject to copyright held by third parties, including without limitation: Copyright DataDirect Technologies. All rights reserved. Copyright © Sun Microsystems. All rights reserved. Copyright © RSA Security Inc. All Rights Reserved. Copyright © Ordinal Technology Corp. All rights reserved. Copyright © Aandacht c.v. All rights reserved. Copyright Genivia, Inc. All rights reserved. Copyright Isomorphic Software. All rights reserved. Copyright © Meta Integration Technology, Inc. All rights reserved. Copyright © Intalio. All rights reserved. Copyright © Oracle. All rights reserved. Copyright © Adobe Systems Incorporated. All rights reserved. Copyright © DataArt, Inc. All rights reserved. Copyright © ComponentSource. All rights reserved. Copyright © Microsoft Corporation. All rights reserved. Copyright © Rogue Wave Software, Inc. All rights reserved. Copyright © Teradata Corporation. All rights reserved. Copyright © Yahoo! Inc. All rights reserved. Copyright © Glyph & Cog, LLC. All rights reserved. Copyright © Thinkmap, Inc. All rights reserved. Copyright © Clearpace Software Limited. All rights reserved. Copyright © Information Builders, Inc. All rights reserved. Copyright © OSS Nokalva, Inc. All rights reserved. Copyright Edifecs, Inc. All rights reserved. Copyright Cleo Communications, Inc. All rights reserved. Copyright © International Organization for Standardization 1986. All rights reserved. Copyright © ej-technologies GmbH. All rights reserved. Copyright © Jaspersoft Corporation. All rights reserved. Copyright © International Business Machines Corporation. All rights reserved. Copyright © yWorks GmbH. All rights reserved. Copyright © Lucent Technologies. All rights reserved. Copyright © University of Toronto. All rights reserved. Copyright © Daniel Veillard. All rights reserved. Copyright © Unicode, Inc. Copyright IBM Corp. All rights reserved. Copyright © MicroQuill Software Publishing, Inc. All rights reserved. Copyright © PassMark Software Pty Ltd. All rights reserved. Copyright © LogiXML, Inc. All rights reserved. Copyright © 2003-2010 Lorenzi Davide, All rights reserved. Copyright © Red Hat, Inc. All rights reserved. Copyright © The Board of Trustees of the Leland Stanford Junior University. All rights reserved. Copyright © EMC Corporation. All rights reserved. Copyright © Flexera Software. All rights reserved. Copyright © Jinfonet Software. All rights reserved. Copyright © Apple Inc. All rights reserved. Copyright © Telerik Inc. All rights reserved. Copyright © BEA Systems. All rights reserved. Copyright © PDFlib GmbH. All rights reserved. Copyright © Orientation in Objects GmbH. All rights reserved. Copyright © Tanuki Software, Ltd. All rights reserved. Copyright © Ricebridge. All rights reserved. Copyright © Sencha, Inc. All rights reserved. Copyright © Scalable Systems, Inc. All rights reserved. Copyright © jQWidgets. All rights reserved. Copyright © Tableau Software, Inc. All rights reserved. Copyright© MaxMind, Inc. All Rights Reserved. Copyright © TMate Software s.r.o. All rights reserved. Copyright © MapR Technologies Inc. All rights reserved. Copyright © Amazon Corporate LLC. All rights reserved. Copyright © Highsoft. All rights reserved. Copyright © Python Software Foundation. All rights reserved. Copyright © BeOpen.com. All rights reserved. Copyright © CNRI. All rights reserved.

This product includes software developed by the Apache Software Foundation (http://www.apache.org/), and/or other software which is licensed under various versions of the Apache License (the "License"). You may obtain a copy of these Licenses at http://www.apache.org/licenses/. Unless required by applicable law or agreed to in writing, software distributed under these Licenses is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the Licenses for the specific language governing permissions and limitations under the Licenses.

This product includes software which was developed by Mozilla (http://www.mozilla.org/), software copyright The JBoss Group, LLC, all rights reserved; software copyright © 1999-2006 by Bruno Lowagie and Paulo Soares and other software which is licensed under various versions of the GNU Lesser General Public License Agreement, which may be found at http:// www.gnu.org/licenses/lgpl.html. The materials are provided free of charge by Informatica, "as-is", without warranty of any kind, either express or implied, including but not limited to the implied warranties of merchantability and fitness for a particular purpose.

The product includes ACE(TM) and TAO(TM) software copyrighted by Douglas C. Schmidt and his research group at Washington University, University of California, Irvine, and Vanderbilt University, Copyright (©) 1993-2006, all rights reserved.

This product includes software developed by the OpenSSL Project for use in the OpenSSL Toolkit (copyright The OpenSSL Project. All Rights Reserved) and redistribution of this software is subject to terms available at http://www.openssl.org and http://www.openssl.org/source/license.html.

This product includes Curl software which is Copyright 1996-2013, Daniel Stenberg, <daniel@haxx.se>. All Rights Reserved. Permissions and limitations regarding this software are subject to terms available at http://curl.haxx.se/docs/copyright.html. Permission to use, copy, modify, and distribute this software for any purpose with or without fee is hereby granted, provided that the above copyright notice and this permission notice appear in all copies.

The product includes software copyright 2001-2005 (©) MetaStuff, Ltd. All Rights Reserved. Permissions and limitations regarding this software are subject to terms available at http://www.dom4j.org/ license.html.

The product includes software copyright © 2004-2007, The Dojo Foundation. All Rights Reserved. Permissions and limitations regarding this software are subject to terms available at http://dojotoolkit.org/license.

This product includes ICU software which is copyright International Business Machines Corporation and others. All rights reserved. Permissions and limitations regarding this software are subject to terms available at http://source.icu-project.org/repos/icu/icu/trunk/license.html.

This product includes software copyright © 1996-2006 Per Bothner. All rights reserved. Your right to use such materials is set forth in the license which may be found at http:// www.gnu.org/software/ kawa/Software-License.html.

This product includes OSSP UUID software which is Copyright © 2002 Ralf S. Engelschall, Copyright © 2002 The OSSP Project Copyright © 2002 Cable & Wireless Deutschland. Permissions and limitations regarding this software are subject to terms available at http://www.opensource.org/licenses/mit-license.php.

This product includes software developed by Boost (http://www.boost.org/) or under the Boost software license. Permissions and limitations regarding this software are subject to terms available at http:/ /www.boost.org/LICENSE_1_0.txt.

This product includes software copyright © 1997-2007 University of Cambridge. Permissions and limitations regarding this software are subject to terms available at http:// www.pcre.org/license.txt.

This product includes software copyright © 2007 The Eclipse Foundation. All Rights Reserved. Permissions and limitations regarding this software are subject to terms available at http:// www.eclipse.org/org/documents/epl-v10.php and at http://www.eclipse.org/org/documents/edl-v10.php.

This product includes software licensed under the terms at http://www.tcl.tk/software/tcltk/license.html, http://www.bosrup.com/web/overlib/?License, http://www.stlport.org/doc/ license.html, http://asm.ow2.org/license.html, http://www.cryptix.org/LICENSE.TXT, http://hsqldb.org/web/hsqlLicense.html, http://httpunit.sourceforge.net/doc/ license.html, http://jung.sourceforge.net/license.txt , http://www.gzip.org/zlib/zlib_license.html, http://www.openldap.org/software/release/license.html, http://www.libssh2.org, http://slf4j.org/license.html, http://www.sente.ch/software/OpenSourceLicense.html, http://fusesource.com/downloads/license-agreements/fuse-message-broker-v-5-3- license-agreement; http://antlr.org/license.html; http://aopalliance.sourceforge.net/; http://www.bouncycastle.org/licence.html; http://www.jgraph.com/jgraphdownload.html; http://www.jcraft.com/jsch/LICENSE.txt; http://jotm.objectweb.org/bsd_license.html; . http://www.w3.org/Consortium/Legal/2002/copyright-software-20021231; http://www.slf4j.org/license.html; http://nanoxml.sourceforge.net/orig/copyright.html; http://www.json.org/license.html; http://forge.ow2.org/projects/javaservice/, http://www.postgresql.org/about/licence.html, http://www.sqlite.org/copyright.html, http://www.tcl.tk/software/tcltk/license.html, http://www.jaxen.org/faq.html, http://www.jdom.org/docs/faq.html, http://www.slf4j.org/license.html; http://www.iodbc.org/dataspace/iodbc/wiki/iODBC/License; http://www.keplerproject.org/md5/license.html; http://www.toedter.com/en/jcalendar/license.html; http://www.edankert.com/bounce/index.html; http://www.net-snmp.org/about/license.html; http://www.openmdx.org/#FAQ; http://www.php.net/license/3_01.txt; http://srp.stanford.edu/license.txt; http://www.schneier.com/blowfish.html; http://www.jmock.org/license.html; http://xsom.java.net; http://benalman.com/about/license/; https://github.com/CreateJS/EaselJS/blob/master/src/easeljs/display/Bitmap.js; http://www.h2database.com/html/license.html#summary; http://jsoncpp.sourceforge.net/LICENSE; http://jdbc.postgresql.org/license.html; http://protobuf.googlecode.com/svn/trunk/src/google/protobuf/descriptor.proto; https://github.com/rantav/hector/blob/master/LICENSE; http://web.mit.edu/Kerberos/krb5-current/doc/mitK5license.html; http://jibx.sourceforge.net/jibx-license.html; https://github.com/lyokato/libgeohash/blob/master/LICENSE; https://github.com/hjiang/jsonxx/blob/master/LICENSE; https://code.google.com/p/lz4/; https://github.com/jedisct1/libsodium/blob/master/LICENSE; http://one-jar.sourceforge.net/index.php?page=documents&file=license; https://github.com/EsotericSoftware/kryo/blob/master/license.txt; http://www.scala-lang.org/license.html; https://github.com/tinkerpop/blueprints/blob/master/LICENSE.txt; http://gee.cs.oswego.edu/dl/classes/EDU/oswego/cs/dl/util/concurrent/intro.html; https://aws.amazon.com/asl/; https://github.com/twbs/bootstrap/blob/master/LICENSE; https://sourceforge.net/p/xmlunit/code/HEAD/tree/trunk/LICENSE.txt; https://github.com/documentcloud/underscore-contrib/blob/master/LICENSE, and https://github.com/apache/hbase/blob/master/LICENSE.txt.

This product includes software licensed under the Academic Free License (http://www.opensource.org/licenses/afl-3.0.php), the Common Development and Distribution License (http://www.opensource.org/licenses/cddl1.php) the Common Public License (http://www.opensource.org/licenses/cpl1.0.php), the Sun Binary Code License Agreement Supplemental License Terms, the BSD License (http:// www.opensource.org/licenses/bsd-license.php), the new BSD License (http://opensource.org/licenses/BSD-3-Clause), the MIT License (http://www.opensource.org/licenses/mit-license.php), the Artistic License (http://www.opensource.org/licenses/artistic-license-1.0) and the Initial Developer's Public License Version 1.0 (http://www.firebirdsql.org/en/initial-developer-s-public-license-version-1-0/).

This product includes software copyright © 2003-2006 Joe Walnes, 2006-2007 XStream Committers. All rights reserved. Permissions and limitations regarding this software are subject to terms available at http://xstream.codehaus.org/license.html. This product includes software developed by the Indiana University Extreme! Lab. For further information please visit http://www.extreme.indiana.edu/.

This product includes software Copyright (c) 2013 Frank Balluffi and Markus Moeller. All rights reserved. Permissions and limitations regarding this software are subject to terms of the MIT license.

See patents at https://www.informatica.com/legal/patents.html.

DISCLAIMER: Informatica LLC provides this documentation "as is" without warranty of any kind, either express or implied, including, but not limited to, the implied warranties of noninfringement, merchantability, or use for a particular purpose. Informatica LLC does not warrant that this software or documentation is error free. The information provided in this software or documentation may include technical inaccuracies or typographical errors. The information in this software and documentation is subject to change at any time without notice.

NOTICES

This Informatica product (the "Software") includes certain drivers (the "DataDirect Drivers") from DataDirect Technologies, an operating company of Progress Software Corporation ("DataDirect") which are subject to the following terms and conditions:

1. THE DATADIRECT DRIVERS ARE PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT.
2. IN NO EVENT WILL DATADIRECT OR ITS THIRD PARTY SUPPLIERS BE LIABLE TO THE END-USER CUSTOMER FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL OR OTHER DAMAGES ARISING OUT OF THE USE OF THE ODBC DRIVERS, WHETHER OR NOT INFORMED OF THE POSSIBILITIES OF DAMAGES IN ADVANCE. THESE LIMITATIONS APPLY TO ALL CAUSES OF ACTION, INCLUDING, WITHOUT LIMITATION, BREACH OF CONTRACT, BREACH OF WARRANTY, NEGLIGENCE, STRICT LIABILITY, MISREPRESENTATION AND OTHER TORTS.

Publication Date: 2025-08-30

Table of Contents

Preface 5

Chapter 1: Introduction to AI Agent with Multifunction Calling using Google Vertex AI recipe 6

AI Agent with Multifunction Calling using Google Vertex AI recipe contents 6

AI Agent with Multifunction Calling using Google Vertex AI recipe assets 7

Chapter 2: Using the AI Agent with Multifunction Calling using Google Vertex AI recipe 8

Step 1. Copy and access the recipe 8

Step 2. Publish the Generic Connector service connector 9

Step 3. Configure and publish the Salesforce connection 9

Step 4. Configure and publish the ServiceNow connection 10

Step 5. Configure and publish the Google Vertex AI connection 10

Step 6. Publish the GenericConnection connection 11

Step 7. Configure and publish the processes 11

Step 8. Publish and run the guide 11

Test AI Agent Multifunction Calling using Google Vertex AI 15

Preface

Use *AI Agent with Multifunction Calling using Google Vertex AI* to learn how to leverage Google Vertex AI's function calling to dynamically generate, orchestrate, and run Application Integration processes based on user queries and seamlessly pass outputs between steps. This guide assumes that you have an understanding of the Salesforce Connector, ServiceNow Connector, and Google Vertex AI Connector concepts.

CHAPTER 1

Introduction to AI Agent with Multifunction Calling using Google Vertex AI recipe

The AI Agent with Multifunction Calling using Google Vertex AI recipe is based on REST and SOAP APIs. The recipe offers a custom AI agent framework that leverages Google Vertex AI's function calling to dynamically generate, orchestrate, and run Application Integration processes based on user queries and seamlessly pass outputs between steps.

The recipe allows you to specify the user prompt and enter the function declaration in the JSON format. The recipe uses a guide to query other processes based on a function call response.

The process sends a request to Google Vertex AI with the specified function declaration and user prompt. Google Vertex AI recognizes the query, extracts key variables and values, and returns them in the appropriate format. Based on the response received from the function call, the LLM summarizes and describes the extracted parameters.

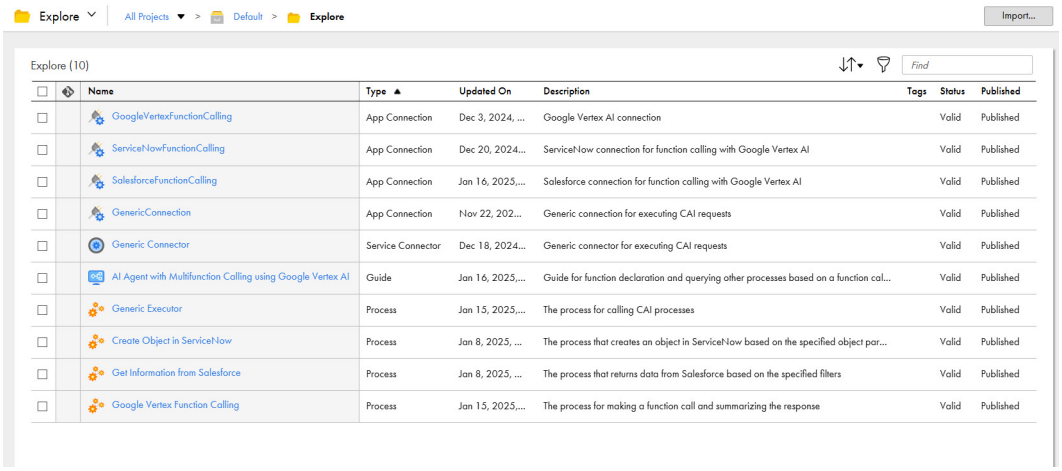
The user confirms the accuracy of the function call response to make the call to the Application Integration process or doesn't make a function call again. The process requests the first Application Integration process with parameters based on the function-calling response. Based on the first process response, a new request is made to the LLM. Google Vertex AI recognizes the service from which the response came and remaps the fields to the target service according to the instructions. The process makes the next request to the Application Integration recipe based on the function-calling response. The user then receives a message about the success of the recipe call and a response from the second Application Integration recipe.

The recipe contains two processes to perform function calling. You can increase the number of processes or modify the existing processes by updating the recipe parameters.

AI Agent with Multifunction Calling using Google Vertex AI recipe contents

The AI Agent with Multifunction Calling using Google Vertex AI recipe contains a service connector, app connections, processes, and a guide.

The following image shows the assets that the AI Agent with Multifunction Calling using Google Vertex AI recipe package contains:



AI Agent with Multifunction Calling using Google Vertex AI recipe assets

The following table lists the assets that the AI Agent with Multifunction Calling using Google Vertex AI recipe package contains:

Asset Name	Asset Type	Description
Generic Connector	Service connector	Generic connector for executing Application Integration requests.
GoogleVertexFunctionCalling	App connection	Google Vertex AI connection.
ServiceNowFunctionCalling	App connection	ServiceNow connection for function calling with Google Vertex AI.
SalesforceFunctionCalling	App connection	Salesforce connection for function calling with Google Vertex AI.
GenericConnection	App connection	Generic connection for executing Application Integration requests.
Generic Executor	Process	Process to call other Application Integration processes.
Create Object in ServiceNow	Process	Process that creates an object in ServiceNow based on the specified object parameters.
Get Information from Salesforce	Process	Process that returns data from Salesforce based on the specified filters.
Google Vertex Function Calling	Process	Process for making a function call and summarizing the response.
AI Agent with Multifunction Calling using Google Vertex AI	Guide	Guide for function declaration and querying other processes based on a function call response

Note: You can configure any other processes. The ServiceNow and Salesforce connections and related processes are added to illustrate the recipe.

CHAPTER 2

Using the AI Agent with Multifunction Calling using Google Vertex AI recipe

To use the AI Agent with Multifunction Calling using Google Vertex AI recipe, you must perform the following steps manually:

1. Copy and access the recipe.
2. Publish the Generic Connector service connector.
3. Configure and publish the Salesforce connection.
4. Configure and publish the ServiceNow connection.
5. Configure and publish the Google Vertex AI connection.
6. Publish the GenericConnection connection.
7. Configure and publish the processes.
8. Publish and run the guide.

Step 1. Copy and access the recipe

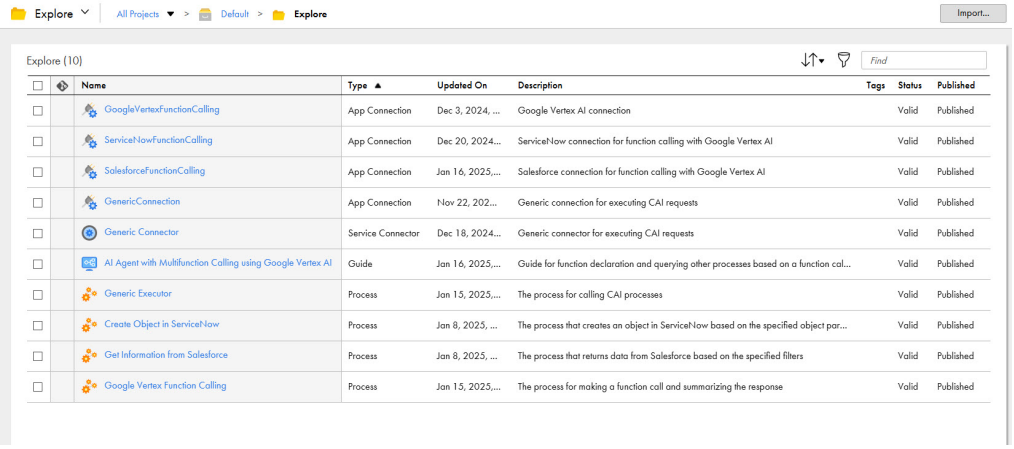
Copy the pre-configured assets in the recipe to a separate project or folder.

1. Open the **AI Agent with Multifunction Calling using Google Vertex AI** recipe and click **Use**.
2. Select the location where you want to copy the recipe, and then click **Continue**.
3. In the **Copying the recipe** dialog box, click **OK**.

   It might take some time for the recipe to get copied. You will receive a notification when the recipe is ready for use.
4. After the recipe is copied, click **Explore** to access the recipe content.

5. Navigate to the project or folder where you copied the recipe or enter the recipe name in the **Find** box.

   All the assets in the recipe are displayed as shown in the following image:



Step 2. Publish the Generic Connector service connector

To publish the **Generic Connector** service connector, open the **Generic Connector** service connector and click **Publish**.

Step 3. Configure and publish the Salesforce connection

Configure the authentication, token request URL, and authorization access details to connect to Salesforce, and then publish the SalesforceFunctionCalling connection.

1. Open the **SalesforceFunctionCalling** connection.
2. From the **Run On** list, select **Cloud Server or any Secure Agent**.
3. From the **Authentication Type** list, select **OAuth** and enter values for the following properties:
   - **Authentication URL**. Endpoint used to make OAuth authorization requests to Salesforce.
   - **Token Request URL**. Endpoint used to make OAuth token requests to Salesforce.
   - **Session Duration**. Number of minutes after which the OAuth token expires.
   - **Authorize Access**. Click **Authorize** > enter the Salesforce developer account credentials > click **Allow**.

   The OAuth authentication process starts. You can check the current authorization status in the **Authorization Status** property.
4. Save and publish the connection.

Step 4. Configure and publish the ServiceNow connection

Configure the client details, user details, service URL, and grand type to connect to the ServiceNow instance, and then publish the ServiceNowFunctionCalling connection.

1. Open the **ServiceNowFunctionCalling** connection.
2. From the **Type** list, select **ServiceNow**.
3. From the **Run On** list, select **Cloud Server or any Secure Agent**.
4. In the **Connection Properties** section, enter values for the following properties:

Property	Description
Client_ID	ServiceNow client ID to generate a valid access and refresh token. Enter the client ID that you generated under **System OAuth** > **Application Registry** in ServiceNow.
Client_Secret	ServiceNow client secret that you generated under **System OAuth** > **Application Registry** in ServiceNow.
User_Name	ServiceNow user name with the security_admin role to generate client credentials in the ServiceNow instance.
Password	Password associated with the ServiceNow user account.
Service_URL	URL to access the ServiceNow instance.
Grant_Type	Grant type that the ServiceNow instance uses to get an access token for third-party client authorization. Enter the value as **password**.

5. Save and publish the connection.

Step 5. Configure and publish the Google Vertex AI connection

Configure the project ID, private key ID, private key, client email, and location to connect to Google Vertex AI, and then publish the GoogleVertexFunctionCalling connection.

1. Open the **GoogleVertexFunctionCalling** connection.
2. In the **Connection Properties** section, enter values for the following properties:

Property	Description
Project_ID	The project ID to generate a valid access token.
Location	The region to process the request.

Property	Description
Private_Key_ID	The private key ID associated with the Google Vertex AI service account.
Client_Email	The email address associated with the Google Vertex AI service account.
Private_Key	The Google Vertex AI private key associated with the service account. Enter the PKCS1 certificate as a Base64-encoded string in the following format: `-----BEGIN PRIVATE KEY----- ..... n-----END PRIVATE KEY-----`

3. Save and publish the connection.

Step 6. Publish the GenericConnection connection

To publish the **GenericConnection** connection, open the **GenericConnection** connection and click **Publish**.

Step 7. Configure and publish the processes

Configure the system instructions and publish the processes.

1. To publish the following processes, click **Actions** in the row that contains the process and select **Publish**.
   - **Get Information from Salesforce**
   - **Create Object in ServiceNow**
   - **Generic Executor**
2. Open the **Google Vertex Function Calling** process.
3. On the **Assignments** tab of the **Prepare Function Call PO** step, you can optionally update the system instructions in the **Function_Call_PO** field using the Expression Editor.

   The system instructions explain to the LLM the order in which the function is executed and the general behavior of the model. To add more processes, you must change the LLM behavior and explain how the LLM can create the order of function calls.
4. Save and publish the process.

Step 8. Publish and run the guide

Configure the Google Vertex AI LLM model version, execution queue count, and process URLs of the processes you want to run, and then publish and run the guide.

1. Open the **AI Agent with Multifunction Calling using Google Vertex AI** guide.
2. On the **Start** tab of the Start step, ensure that the **Run As** field is set to **Current User**.

3. On the **Assignments** tab of the **Set Flow Configuration** step, enter values for the following fields:
    a. In the **Generation_Config** field, enter the prompt instructions using the Expression Editor, as shown in the following sample code:

```
<generationConfig>
    <temperature>0.2</temperature>
    <maxOutputTokens>1024</maxOutputTokens>
    <topP>0.8</topP>
</generationConfig>
```

    For the **Generation_Config** field, enter values for the following properties:

Property	Description
temperature	Controls the randomness of the model's output. A lower value close to 0 makes the output more deterministic, while a higher value close to 1 increases randomness and creativity. For example, if `temperature` is set to 0.5, the model balances between deterministic and creative outputs.
maxOutputTokens	Defines the maximum number of tokens that the model can generate in its response. Setting a limit ensures that the response is concise and fits within the desired length constraints.
topP	Determines the cumulative probability threshold for token selection. The model considers the smallest set of tokens whose cumulative probability meets or exceeds `topP`. For example, if `topP` is set to 0.1, the model considers only the top 10% most probable tokens at each step.

    b. In the **LLM_Model** field, enter the model ID of the LLM model that you want to use. The model ID is set to **gemini-1.5-pro-002**, by default.
    c. In the **Execution_Queue_Count** field, enter the minimum number of processes to be executed. The number can't be less than 1. The default value is set to **2**.

    Note: The maximum number of processes is limited by the number of declared functions. If the field value is not set, the default value will be the number of declared functions.For example, if you have two processes and two declared functions, but you need to call only one process, set the Execution_Queue_Count field value to 1 and the LLM will define the first declared function that meets the system instruction and the corresponding process will be triggered. If you have two processes and three declared functions and you need to call all the processes, set the Execution_Queue_Count field value to 2, and the LLM will define two processes in order according to the system instruction and the corresponding processes will be triggered.
    d. In the **Function_Declaration** field, access the prompt instruction and replace **<Service_URL_of_the_Application_Integration_Process>** with the process URLs of the processes that you want to execute using the Expression Editor, as shown in the following sample code:

```
<tools>
    <functionDeclarations>
        <name>Get_Salesforce_Object</name>
        <description>The function recognizes standard Salesforce objects and
ensures they align strictly with Salesforce documentation. Based on the user's
query, it constructs accurate filters to isolate relevant data and retrieves the
corresponding values for these filters. This process guarantees compliance with
Salesforce's data structure and optimizes the query results for precision and
relevance.</description>
        <parameters>
            <type>object</type>
            <properties>
                <Process_URL>
```

```
                    <type>string</type>
                    <description>Fixed URL to execute process</description>
                    <enum><Service_URL_of_the_Application_Integration_Process></
enum>
                </Process_URL>
                <Payload>
                    <type>array</type>
                    <description>Structured payload for processes</description>
                    <items>
                        <description>Fields in payload for processes</
description>
                        <type>object</type>
                        <properties>
                            <Entity_Type>
                                <type>string</type>
                                <description>Standard Salesforce objects (e.g.,
Account, Contact) represent key business information. These built-in data tables
provide core CRM functionality.</description>
                            </Entity_Type>
                            <Filter_Name>
                                <type>string</type>
                                <description>The field name by which the
standard Salesforce objects will be filtered. This field name must match with
the corresponding field names defined in Salesforce's documentation to ensure
accurate and compliant data filtering.</description>
                            </Filter_Name>
                            <Filter_Data>
                                <type>string</type>
                                <description>The value associated with the
specified Filter_Name, which is required when Filter_Name is present. This
variable filters records for standard Salesforce objects, ensuring precise data
selection in line with Salesforce's structure.</description>
                            </Filter_Data>
                        </properties>
                        <required>Entity_Type</required>
                        <required>Filter_Name</required>
                        <required>Filter_Data</required>
                    </items>
                </Payload>
            </properties>
            <required>Process_URL</required>
            <required>Payload</required>
        </parameters>
    </functionDeclarations>
    <functionDeclarations>
        <name>Create_ServiceNow_Object</name>
        <description>This function processes JSON input and is triggered only
when a valid JSON structure is provided. It maps fields into a ServiceNow-
compatible payload, following the specifications and structure outlined in the
ServiceNow documentation. The function extracts relevant data, such as user
details or system information, and transforms it into a format that ServiceNow
can accept, ensuring proper field mappings and adherence to the required schema
conventions.</description>
        <parameters>
            <type>object</type>
            <properties>
                <Process_URL>
                    <type>string</type>
                    <description>Fixed URL to execute process</description>
                    <enum><Service_URL_of_the_Application_Integration_Process></
enum>
                </Process_URL>
                <Payload>
                    <type>array</type>
                    <description>Structured payload for processes</description>
                    <items>
                        <description>Fields in payload for processes</
description>
```

```
                        <type>object</type>
                        <properties>
                            <Table_Name>
                                <type>string</type>
                                <description>Specifies the name of a particular
table in the ServiceNow system used for data storage. This variable is used to
identify the target table for operations, such as data retrieval, insertion, or
modification within the system. For the case table in Salesforce, use the
corresponding incident table in ServiceNow, for the account table, use the
core_company table, for the contact table, use the sys_user table, for the
opportunity table, use the x_mas_opportunity_opportunity table.</description>
                            </Table_Name>
                            <Object>
                            <type>string</type>
                            <description>When handling a response, the first
step is identifying the key fields required for integration with ServiceNow. For
the Case table, map number to number, subject to short_description, contact to
u_name, description to description. For the Opportunity table, map stage to
isclosed and iswon and define the transformation logic, map amount to amount,
name to opportunity_name. For the Contact table, map first_name to first_name,
last_name to last_name, email to email, mailing_address to street, mailing_city
to city, mailing_state to state, mailing_zip to zip, and set the country field.
For the Account table, map name to name, phone to phone, billing_address to
street, billing_city to city, billing_state to state, billing_zip to zip, and
set the country field. Use only the transformation examples and fields above to
transform fields correctly. If the example does not have a field and the prompt
does, skip it. After the fields are mapped and converted, the data must be
structured into a valid JSON format that meets the ServiceNow API standards.
This JSON object must contain all the required fields in the correct structure
and format. The result must be a clean, correct JSON object containing only the
required data fields, with no additional text, explanations, or extraneous
characters.</description>
                            </Object>
                        </properties>
                        <required>Table_Name</required>
                        <required>Object</required>
                    </items>
                </Payload>
            </properties>
            <required>Process_URL</required>
            <required>Payload</required>
        </parameters>
    </functionDeclarations>
</tools>
```

4. Save and publish the guide.
5. On the **Actions** menu, click **Run**. Alternatively, you can copy the execution URL from the **Properties Details** dialog box to run the guide.
6. On the **Instructions** page, enter the following values:
   - In the **Enter a prompt below** field, enter your question based on the function declaration.
   - Optionally, in the **Function declaration** field, enter the custom function declaration. By default, the function declaration value is set to Get_Salesforce_Object and Create_ServiceNow_Object in the **Set Flow Configuration** step of the guide. This executes the **Get Information from Salesforce** and **Create Object in ServiceNow** processes that are available in the recipe.
   - Select the function calling confirmation as **No** if you don't want to receive a summary of the function call response and omit the confirmation step before the function is executed. By default, the function calling confirmation is selected as **Yes**.
7. Click **Continue**.
   You will receive a summary of the function call request in response if the function call is generated. Otherwise, an error message appears.

8. Click **Confirm** to confirm the summary of the function call response and make a call to the CAI process or click **Try again**.
   You will receive a response that contains the process call status, the process execution status, and the process response.

Note: If the process you are calling returns any response, you will see the response on the Answer screen. However, if the process runs for more than 59 seconds, you will see a message about the successful process call, but the result can be viewed in Application Integration Console.

You can also use the embed code to embed the guide into an HTML document of a third-party application.

Test AI Agent Multifunction Calling using Google Vertex AI

You can test the AI Agent Multifunction Calling using Google Vertex AI recipe with the function declaration value that is set as default in the **Set Flow Configuration** step of the **AI Agent with Multifunction Calling using Google Vertex AI** guide.

The default value applies to and can execute the **Get Information from Salesforce** and **Create Object in ServiceNow** processes that are available in this recipe.

For example, in the **AI Agent with Multifunction Calling using Google Vertex AI** guide, you can enter one of the following prompts for the correct function call for these recipes:

- `Create a new Salesforce case with the Reason 'Breakdown' in the ServiceNow system`
- `Create a new Salesforce opportunity with the name 'Dickenson Mobile Generators' in the ServiceNow system`

The function name must start with a letter or an underscore, and can contain alphanumeric characters, underscores (_), periods (.), and hyphens (-). For more information, see Google Vertex AI documentation.

The structure of the **Function_Declaration** is important for further execution of the request to the Application Integration process. To set your **Function_Declaration** in the guide, you must specify it in the JSON format as shown in the following example:

```
{
  "tools": [{
    "functionDeclarations": [
      {
        "name": "Get_Salesforce_Object",
        "description": "The function recognizes standard Salesforce objects and ensures
they align strictly with Salesforce documentation. Based on the user's query, it
constructs accurate filters to isolate relevant data and retrieves the corresponding
values for these filters. This process guarantees compliance with Salesforce's data
structure and optimizes the query results for precision and relevance.",
        "parameters": {
          "type": "object",
          "properties": {
            "Process_URL": {
              "type": "string",
              "description": "Fixed URL to execute process",
              "enum": "<Service_URL_of_the_Application_Integration_Process>"
            },
            "Payload": {
              "type": "array",
              "description": "Structured payload for processes",
              "items": {
                "description": "Fields in payload for processes",
                "type": "object",
```

```
                "properties": {
                  "Entity_Type": {
                    "type": "string",
                    "description": "Standard Salesforce objects (e.g., Account, Contact)
represent key business information. These built-in data tables provide core CRM
functionality."
                  },
                  "Filter_Name": {
                    "type": "string",
                    "description": "The field name by which the standard Salesforce
objects will be filtered. This field name must match with the corresponding field names
defined in the Salesforce documentation to ensure accurate and compliant data filtering."
                  },
                  "Filter_Data": {
                    "type": "string",
                    "description": "The value associated with the specified Filter_Name,
which is required when Filter_Name is present. This variable filters records for
standard Salesforce objects, ensuring precise data selection in line with the Salesforce
structure."
                  }
                },
                "required": [
                  "Entity_Type",
                  "Filter_Name",
                  "Filter_Data"
                ]
              }
            }
          },
          "required": [
            "Process_URL",
            "Payload"
          ]
        }
      },
      {
        "name": "Create_ServiceNow_Object",
        "description": "This function processes JSON input and is triggered only when a
valid JSON structure is provided. It maps fields into a ServiceNow-compatible payload,
following the specifications and structure outlined in the ServiceNow documentation. The
function extracts relevant data, such as user details or system information, and
transforms it into a format that ServiceNow can accept, ensuring proper field mappings
and adherence to the required schema conventions.",
        "parameters": {
          "type": "object",
          "properties": {
            "Process_URL": {
              "type": "string",
              "description": "Fixed URL to execute process",
              "enum": "<Service_URL_of_the_Application_Integration_Process>"
            },
            "Payload": {
              "type": "array",
              "description": "Structured payload for processes",
              "items": {
                "description": "Fields in payload for processes",
                "type": "object",
                "properties": {
                  "Table_Name": {
                    "type": "string",
                    "description": "Specifies the name of a particular table used for
data storage in the ServiceNow system. This variable is used to identify the target
table for operations, such as data retrieval, insertion, or modification within the
system. For the case table in Salesforce, use the corresponding incident table in
ServiceNow, for the account table, use the core_company table, for the contact table,
use the sys_user table, for the opportunity table, use the x_mas_opportunity_opportunity
table."
                  },
                  "Object": {
                    "type": "string",
                    "description": "When handling a response, the first step is
```

```
identifying the key fields required for integration with ServiceNow. For the Case table,
map number to number, subject to short_description, contact to u_name, description to
description. For the Opportunity table, map stage to isclosed and iswon and define the
transformation logic, map amount to amount, name to opportunity_name. For the Contact
table, map first_name to first_name, last_name to last_name, email to email,
mailing_address to street, mailing_city to city, mailing_state to state, mailing_zip to
zip, and set the country field. For the Account table, map name to name, phone to phone,
billing_address to street, billing_city to city, billing_state to state, billing_zip to
zip, and set the country field. Use only the transformation examples and fields above to
transform fields correctly. If the example does not have a field and the prompt does,
skip it. After the fields are mapped and converted, the data must be structured into a
valid JSON format that meets the ServiceNow API standards. This JSON object must contain
all the required fields in the correct structure and format. The result must be a clean,
correct JSON object containing only the required data fields, with no additional text,
explanations, or extraneous characters."
                  }
                },
                "required": [
                  "Table_Name",
                  "Object"
                ]
              }
            }
          },
          "required": [
            "Process_URL",
            "Payload"
          ]
        }
      }
    ]
  }]
}
```

For testing, you can construct another structure of a Function Declaration as required.